Native Unicode Support in the Unicon Runtime

Jafar Al-Gharaibeh
Clinton Jeffery
Bruce Rennie
Don Ward

Unicon Technical Report: 25

August 2026

Abstract

Unicon strings historically have no concept of Unicode: a string is a byte sequence, and operations like subscripting, length, and concatenation operate on bytes. This report describes a native implementation that gives strings UTF-8 awareness while preserving byte-for-byte performance and memory layout for content that is pure ASCII. The approach – tagging a string's existing descriptor rather than introducing a new allocated representation – is the tagged-qualifier design. The report covers the bit-level representation, compile-time and I/O tagging, Unicode-aware operations, the language API, examples, and remaining work.

Keywords: Unicon, Unicode, UTF-8, strings, runtime, technical report.

Unicon Project
http://unicon.org
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

This report describes native Unicode support in the Unicon runtime: how a string is tagged, how literals and I/O become tagged, which operations interpret positions as codepoints, and what is still byte-oriented. It is formatted as a Unicon Technical Report [1]. section 8 collects short programs by operation. Automated tests live under `tests/unicode/`.

The feature is optional. A 64-bit build without `NoUniconUnicode` reports `Unicode` in `&features`. On 32-bit, or with that switch, the tag macros are no-ops and the suite is skipped.

1.1 1.1 Terminology: Unicode vs. UTF-8

The two words are not interchangeable here.

Unicode is the language-facing idea: codepoints, text semantics, and the feature itself (`&features` reports `Unicode`; `_UNICODE`; `UniconUnicode`; `unicode()`, `uchar()`, `uord()`; `*` / `s[i]` on a tagged string). `unicode(s)` means "treat these bytes as Unicode text," not "encode as Unicode."

UTF-8 is the encoding of the payload [2]: the bytes in string space, `open()` `i`-mode decoding, lead-byte width scans, and well-formedness. ASCII is a subset of UTF-8; a plain (untagged) string is a byte string. `type()` stays `"string"` for both views. The inverse of `unicode(s)`, and the difference between a content test and a tag test, are in section 7.

The runtime names the tag `F_UniQual` / `IsUniQual`. The class library `UTF8`/`UTF8Set` is a separate, byte-walking encoding layer. Native support is the Unicode view of those same bytes.

2 2. Design principle and representation

Unicon strings are qualifiers: a two-word descriptor consisting of a length word (`dword`) and a pointer into string space (`vword.sptr`). There is no allocated block and no indirection beyond the pointer itself. Plain string operations are cheap because of that layout, and ASCII content must keep that cost: the same memory layout, the same macros, and the same execution speed.

The tag reuses bits already present in the descriptor's length word rather than introducing a new block type. A 64-bit word has far more range than

any realistic string length needs, so unused high bits can hold extra state without changing the descriptor's size or a plain string's representation.

2.1 2.1 Bit layout

Available on 64-bit builds (`WordBits == 64`) when compiled with `UniconUnicode` defined (`src/h/config.h`). With the feature disabled, every macro described below reduces to today's unmodified behavior, so no call site needs its own conditional compilation.

```
bit 63       F_Nqual        (existing -- qualifier vs. typed value)
bit 62       F_UniQual      (this string is UTF-8-tagged)
bits 32-61   cp_count       (30 bits, cached codepoint count)
bits 0-31    byte length    (32 bits, ~4.3 GB maximum)
```

Byte length keeps a full 32 bits. Unlike the codepoint count, it has no fallback: Unicon strings are not null-terminated, so if the true length does not fit, there is nothing to recover it from. The codepoint count can always be recomputed by walking the string, so it takes whatever bits remain. In practice that is generous – the sentinel (see below) is reached only by a string near the 4.3 GB byte-length limit that is almost entirely single-byte, which would not be UTF-8-tagged in ordinary use.

The remaining bits hold a cached *codepoint count*, so `*s` is constant-time. The other use of those bits would be a cached *scan position*: a (codepoint index, byte offset) pair for the last lookup, so a nearby subscript can resume without walking from the start. A count helps every size query; there is no cheaper way to answer how many codepoints a string has. A cached position helps only when a string is subscripted repeatedly and nontrivially. Direct measurement showed that for short strings accessed a few times, walking from the start is already competitive with maintaining an index. Typical Unicon strings are short-lived tokens – parsed, compared, and discarded – so the bits go to the count.

A packed-position layout on the same budget (14-bit codepoint index and 16-bit byte offset) was benchmarked and matched a fully allocated index to within a few percent up to roughly sixteen thousand codepoints. The bits

still went to the count. The packed-position layout remains a possible later change (section 9).

2.2 2.2 Core macros

Defined in `src/h/rmacros.h`:

```
StrLen(q)         /* byte length, masked to exclude tag/count bits
↪  */
SetStrLen(q,n)    /* sets byte length; see section 2.3 for why this
↪  clears
                     rather than preserves the other fields */
MakeStr(s,n,dp)   /* StrLoc(*dp) = s; SetStrLen(*dp, n) -- preferred
                     constructor for a fresh string descriptor */
IsUniQual(d)      /* tests the UTF-8 tag */
SetUniQual(d)     /* sets the UTF-8 tag */
CpCount(q)        /* reads the cached codepoint count, or the
↪  sentinel
                     value 0 if none is cached */
SetCpCount(q,n)   /* sets the cached codepoint count */
```

`MakeStr` is the preferred way to fill a new string descriptor: it sets the pointer and then the length in one step, so call sites share one order. `StrLoc` and `SetStrLen` update different words of the descriptor, so swapping those two alone does not affect the tag; the ordering that matters for Unicode is still `SetStrLen` / `MakeStr`, then `SetUniQual`, then `SetCpCount` (section 2.3). `AsgnCStr` is the C-string specialization of the same pattern.

Three shared helpers implement UTF-8 decoding logic used throughout the runtime:

```
uq_lead_width(b)              /* width in bytes of the UTF-8
↪  character
                                 starting at lead byte b */
uq_scan(bytes, len, &ncps)    /* scans a byte range once; returns
```

```
                                whether any multi-byte character
                                ↪  was
                                found, and the total codepoint
                                ↪  count */
uq_seek_cp(bytes, target_cp)  /* returns the byte offset at which
                                codepoint index target_cp begins */
```

All three treat a malformed lead byte as a one-byte unit and continue, rather than raising an error; this lenient behavior is specific to internal scanning and is distinct from the strict validation applied by `uord()` (section 4). I/O uses the same scan through `UqMaybeTagRead(s, status)`, which tags a just-read string when `Fs_Unicode` is set on the handle (section 6).

2.3 2.3 `StrLen` masking and `SetStrLen` overwrite semantics

`StrLen` unconditionally masks out the tag and count bits before returning a byte length. This has to be unconditional rather than applied only where Unicode-awareness is expected, because a tagged string can reach any code path that handles strings generically – `write()`, list and table operations, anything treating a string as an opaque byte sequence – and any of those reading the raw, unmasked word would compute an incorrect length.

`SetStrLen` overwrites the whole length word rather than clearing only the length bits and preserving tag and count. Most call sites are filling a fresh descriptor – a stack local, an uninitialized table slot – whose prior bits are garbage, not a previous string. Preserving them would leak that garbage into the tag and count. A full overwrite (with the incoming length masked so an oversized value cannot bleed into the count field) always produces a deterministic, untagged result, which is what nearly every call site needs.

The consequence is that any code that both sets a length and applies a tag must do so in order: `SetStrLen` (or `MakeStr`, which ends in `SetStrLen`), then `SetUniQual`, then `SetCpCount` if applicable. Reversing this order silently loses the tag. Concatenation (section 5) is the usual place this matters: compute the combined length with `SetStrLen`, then decide whether to tag. Setting the tag first, relying on an earlier whole-descriptor copy, fails because the subsequent `SetStrLen` clears whatever the copy carried.

3 3. Promotion: how a string becomes tagged

A string literal's bytes are fixed at compile time. `icont` scans the literal once and encodes the result in the compiled program, so loading and executing never re-examines those bytes to decide whether they are Unicode content.

Tagging a literal is compile-time only; the runtime does not scan literals. A first-use scan cannot tell "this literal is untagged because a Unicode-aware `icont` found it to be pure ASCII" from "this literal is untagged because the compiler knew nothing about tagging" – both are an unset tag bit. A scan keyed on "not yet tagged" would therefore walk every literal in every program on first execution, including the common case a current `icont` already classified at compile time. Old icode simply runs with every literal untagged, the same as a build without the feature. Recompilation with a current `icont` is how literals become tagged.

3.1 3.1 Compile-time tagging

`src/icont/tsym.c`'s `putlit` registers each literal during compilation. It scans a string literal's already-decoded bytes (escapes have already been resolved) and sets `F_UniQualLit`, defined in `src/icont/tsym.h` and `src/icont/link.h`. That flag occupies the bit immediately above the four existing literal-type flags (`F_IntLit`, `F_RealLit`, `F_StrLit`, `F_CsetLit`). It is a compiler flag, not the runtime's `F_UniQual` bit.

The flag travels through the compiler's intermediate representation the same way the other literal-type flags do; the format is unchanged. At final code emission (`src/icont/lcode.c`'s `uniquallen`), if the flag is set, `F_UniQual` is written into the length value in the compiled program. The interpreter's literal-loading instruction (`Op_Str` in `src/runtime/interp.r`) loads that value as-is, including on the instruction's existing self-patch that rewrites itself after first use to skip recomputing the literal's address. The tag is already correct in the bytecode the first time it is read.

The tagged length is 64 bits wide. The compiler's existing length representation is 32 bits (`int`) in two places: the parameter to the function that emits the final instruction, and the symbol-table field that holds a literal's length. Writing a 64-bit tagged value through either as originally typed would silently truncate it. Only the emit-function parameter is widened, at the point the tagged value is constructed. The symbol-table field and the intermediate file format stay 32-bit; widening the field would have required

changing that file's text serialization.

Compile-time tagging encodes the UTF-8 tag but not a cached codepoint count, which would need the same width change. A compile-time-tagged literal's count is therefore uncached until something computes it. `*s` is still correct – it falls back to a full scan – but is not constant-time until that is addressed (section 9).

4 4. `uchar()` and `uord()`: Unicode-aware character conversion

Unicon's existing `char()` and `ord()` functions convert between an integer ordinal and a single-character string. Both are strictly byte-oriented: `char(i)` accepts only `0 <= i <= 255` and produces a single-byte string; `ord(s)` accepts only a string of exactly one byte. They stay that way. Existing code depends on byte behavior for purposes unrelated to Unicode text – base-256 encodings and lookup tables indexed by raw byte value among them.

Two new functions provide Unicode-aware equivalents:

- `uchar(i)` accepts any Unicode scalar value (`0 <= i <= 0x10FFFF`, excluding the surrogate range `0xD800-0xDFFF`, which are not valid standalone Unicode scalar values) and returns its UTF-8 encoding as a string, tagged when the encoding is genuinely multi-byte.
- `uord(s)` accepts a string and returns its codepoint value, requiring that `s` consist of exactly one well-formed UTF-8 encoded codepoint – correct length for its lead byte, correctly formed continuation bytes, no overlong encoding, and no surrogate value. Unlike the lenient byte-width detection used internally for scanning (section 2.2), `uord()` is strict: malformed or wrong-length input raises a runtime error rather than failing silently, matching the convention already used by `ord()` for its own precondition violations.

Both are registered as ordinary built-in functions (`src/h/fdefs.h`) and implemented in `src/runtime/fmisc.r` alongside `char()` and `ord()`. `uord(uchar(i)) = i` holds across the Basic Multilingual Plane and the supplementary planes, including four-byte-encoded codepoints.

5 5. Concatenation

The `||` operator (`src/runtime/ocat.r`) has three internal implementation paths, corresponding to different memory-layout optimizations (zero-copy when operands are already adjacent in string space, in-place extension when the left operand is the most recently allocated string, and a general copying path otherwise). All three propagate both the UTF-8 tag and the cached codepoint count to their result: the tag is set if either operand is tagged; the codepoint count is computed as the sum of each operand's contribution, where an untagged operand's own byte length stands in for its codepoint count (valid, since an untagged string is by construction pure ASCII), and a tagged operand contributes its cached count if one is available. If either operand's count is not available (uncached or otherwise unknown), the result's count is left uncached rather than guessed, and a subsequent size query on the result falls back to a full scan.

Per section 2.3, all three paths must apply tagging after computing the result length via `SetStrLen`, not before.

6 6. Unscanned input: files and sockets

Data arriving through `read()`, `reads()`, and equivalent socket operations is not scanned or tagged automatically, regardless of content. Read operations process new bytes on every call, with no repeated-execution structure to amortize a scan against. A mandatory scan would charge every I/O call in every program, whether or not the data ever contains non-ASCII. Measured in isolation, a combined copy-and-scan was roughly sixteen times slower than copy-only at a typical line-read size (128 bytes), and remained an order of magnitude or more slower from tens of bytes to several kilobytes: a modern `memcpy` is substantially better optimized than an equivalent scanning loop. ASCII content should not pay that cost, so reads stay copy-only unless the program asks for tagging.

That choice matters for concatenation (section 5). Every operation that examines the tag treats an untagged string as pure ASCII, including codepoint-count propagation on `||`. That holds for every string the runtime produces internally. It does not hold for external UTF-8 that was never scanned. A program that reads such content and uses Unicode-aware operations without tagging first sees byte-oriented results – no data is lost or corrupted, but the

view is not Unicode until one of the mechanisms below tags it.

Three mechanisms tag content when the program wants it:

- `unicode(s)` (`src/runtime/omisc.r`), an explicit, on-demand function. If `s` is untagged, it is scanned; if the scan finds non-ASCII content, the result is tagged and its codepoint count cached. If `s` is already tagged but has no cached count (the common case for a compile-time-tagged literal, section 3.1), the count is computed and cached. If `s` is pure ASCII, it is returned unchanged. `string(s)` is not the inverse; see section 7.

- **The `i` mode character to `open()`**, which sets `Fs_Unicode` (file-status bit `040`, the slot previously marked available in `src/h/rmacros.h`) on the handle. Subsequent `read()` / `reads()` calls then run `UqMaybeTagRead` and tag their results when the payload is multi-byte. This is implemented at the file, socket, messaging, and SSH channel return points [3]. Pseudo-terminal *reads* will tag if the bit is set; `open()` itself still cannot carry `i` onto a pty handle (section 9). SFTP open rewrites the status word and likewise drops the bit.

- **A global default** for `open()` when the caller omits a mode is not implemented.

Adding a new file-status flag to `open()`'s mode-character parsing is not enough to make the flag usable on every connection type. Socket and messaging connections are each validated against an explicit allowlist of permitted flag combinations (`src/runtime/fsys.r`, `if (status & ~(...)) runerr(...)`), independent of the character-parsing switch, so a new flag must be added to each relevant allowlist or it is silently rejected for that connection type. Paths that *assign* `status` rather than `|=` it – SFTP and the pty identity check – must also preserve `Fs_Unicode`, or the mode character is a no-op even though parsing accepted it.

7 7. Language API: views, conversion, and tests

`type()` is `"string"` for both views. There is no `"unicode"` type: `type(x) == "string"`, and `||` / `string()` treat tagged values as strings. ASCII is never tagged –

`unicode("hello")` is a no-op – and on ASCII, `*s`, `s[i]`, and scanning already match the text view.

`unicode(s)` is the text view: `*` and `[]` are codepoints. The inverse would return the same UTF-8 bytes with an untagged descriptor, so `*` and `[]` mean bytes. That operation is not implemented; a descriptor copy plus `SetStrLen` is sufficient. Until then, the practical way to drop the tag is to write the bytes and read them back without mode `i`.

7.1 7.1 `string()` is not the inverse

`string(x)` converts to string or fails. On a string it is identity, including the tag. `string(a, b, ...)` concatenates and keeps the tag, the same as `||`. That identity matters: `s := string(x) | fail` is the usual coerce, used in `uni/lib/format.icn`, GUI text fields, XML `string(e)`, and the IPL. Stripping the tag would make later `*` / `[]` byte-oriented on `"café"`. `if string(x) then` would still succeed as a type test if the tag were dropped; `if s := string(x) then` would bind the byte view.

`bytes(s)` would read as storage size, `utf8(s)` as encode (the opposite of `unicode(s)`), and `plain(s)` is uninformative, so none of those is a good name for the inverse. If the inverse exists, a builtin `isunicode(s)` is unnecessary; see section 7.2.

7.2 7.2 Content test vs. tag test

These are different questions. Below, `foo` stands for the inverse of `unicode`.

Question	isunicode (content)	istagged (descriptor)
Asks	text view ~= byte view?	is `F_UniQual` set on this value?
Form	`*unicode(s) ~= *foo(s)`	must read the bit; no derived form
`*unicode(s)`	codepoints (bytes if ASCII)	
`*foo(s)`	always bytes	

`s`	`*unicode(s)`	`*foo(s)`	content `~=`	tagged?
`"hello"`	5	5	no	no

s	*unicode(s)	*foo(s)	content ~=	tagged?
"café" (literal)	4	5	yes	yes
café bytes, no i	4	5	yes	no

The last two rows have the same * comparison and a different tag. A content test is not a tag test. Inverse plus * answers the content question. A tag predicate is needed only if a program must ask "am I in the text view?" without assigning unicode(s).

8 8. Examples

These examples assume a Unicon build with Unicode in &features. Non-ASCII literals are tagged at compile time, so "café" already has a text view: *s and s[i] are codepoints. The programs under tests/unicode/ are the executable form of this section; languages.icn is a longer script sample.

8.1 8.1 Length and subscript

*s is 4, not 5. s[4] is the letter é, not a continuation byte:

```
procedure main()
   local s, i
   s := "café"
   write("*s = ", *s)
   every i := 1 to *s do
      write("s[", i, "] = ", s[i])
end
```

Output:

```
*s = 4
s[1] = c
s[2] = a
s[3] = f
s[4] = é
```

`"hello"` is untagged ASCII, so `*` and `[]` still mean bytes, which for ASCII are the same as codepoints.

8.2 `unicode()`, `uchar()`, and `uord()` ——————————————————

`unicode(s)` tags non-ASCII UTF-8. `uchar` / `uord` are the codepoint pair; `char` / `ord` stay byte-oriented (section 4):

```
procedure main()
   write(*unicode("hello"))
   write(*unicode("café"))
   write(unicode("café")[4])
   write(uord(uchar(233)))
   write(*uchar(233))
   write(char(65), " ", ord("A"))
end
```

Output:

```
5
4
é
233
1
A 65
```

`uord` on more than one codepoint is error 205.

8.2 8.3 Concatenation

`||` tags the result if either operand is tagged, and adds cached codepoint counts when both are known (section 5):

```
procedure main()
   write(*("café" || "naïve"))
   write(*("café" || "hello"))
   write(*("hello" || "café"))
end
```

Output:

```
9
9
9
```

`string(a, b, ...)` concatenates too and **keeps** the tag, the same as `||`. It is not a byte-view switch (section 7.1).

8.3 8.4 Scanning

On a tagged subject, `move`, `tab`, and `pos` use codepoint positions:

```
procedure main()
   &subject := "café naïve"
   &pos := 1
   write(move(4))
   &pos := 1
   write(tab(5))
   &pos := 1
   write(pos(1))
end
```

Output:

```
café
café
1
```

8.4 8.5 `open()` mode `i`

Without `i`, `reads()` is untagged (byte `*`). With `i`, the same UTF-8 is tagged (section 6):

```
procedure main()
   local f, s
   f := open("uio.dat", "wu") | stop("open")
   writes(f, "café")
   close(f)

   f := open("uio.dat", "r") | stop("open")
   s := reads(f, 100)
   close(f)
   write(*s)

   f := open("uio.dat", "ri") | stop("open")
   s := reads(f, 100)
   close(f)
   write(*s)
   write(s[4])
   remove("uio.dat")
end
```

Output:

```
5
4
é
```

8.5 8.6 Content vs. tag

Untagged UTF-8 from a file, or built from `char()` bytes, has byte `*`. `unicode()` is the text view. The two disagree exactly when the payload is multi-byte (section 7.2):

```
procedure main()
   local raw
   raw := char(195) || char(169)    # UTF-8 of é, untagged
   write(*raw)
   write(*unicode(raw))
   write(*string(unicode(raw)))
end
```

Output:

```
2
1
1
```

`string(s)` on `"café"` also keeps the tag: `*string(s) = *s`.

9 9. Status and future work

The following are Unicode-aware. Subscripting (`s[i]`) and section extraction (`s[i:j]`) tag the result only when the extracted range is genuinely

multi-byte, not merely because the source was tagged – a slice of a tagged string can itself be ASCII. Extraction from a named variable uses a separate path (`src/runtime/cnv.r`'s trapped substring variable) that applies the same rule. Element generation (`!s`) and random selection (`?s`) yield one complete UTF-8 character on a tagged string, matching `s[i]`. Substring assignment (`s[i] := x` and `s[i:j] := x`) uses the same trapped-variable mechanism; the result's tag and codepoint count are recomputed by scanning the assembled string rather than inheriting either the prefix or the suffix tag. `move`, `tab`, and `pos` interpret positions as codepoints for a tagged `&subject`; `cvpos` already treats units abstractly, so only the conversion from abstract position to byte offset needed a tagged branch. `*s`, `||`, `unicode()`, `uchar()`, and `uord()` are as described above.

`many` walks a tagged subject by codepoint, so its positions are codepoint-based. It cannot test a multi-byte codepoint against the character-set argument: a plain cset represents only 0-255. On a multi-byte codepoint it stops without calling the membership test, including when the cset contains that encoding's raw byte values, so continuation bytes are never treated as independent ASCII characters. A codepoint-range set type (below) is what would make `many` match non-ASCII content.

`find` and `match` compare strings, not csets. The internal byte comparison is unchanged: UTF-8 is self-synchronizing, so an exact byte match that begins and ends at codepoint boundaries is already a codepoint-level match. Bounds and returned positions mean codepoints for a tagged subject. `find`, as a generator, walks one codepoint at a time and tests the byte comparison at each codepoint boundary rather than at each byte offset.

Table and set key equality, lookup, and membership ignore the tag. Hashing and equality use `StrLen`-masked byte content (section 2.3), so a tagged literal, a byte-identical untagged string from a file, and a string tagged with `unicode()` compare equal and are interchangeable as keys.

`reverse` cannot be a byte-for-byte reversal: that is correct for ASCII but breaks the lead-byte/continuation-byte structure of multi-byte UTF-8. The implementation reverses codepoint order while leaving each codepoint's own bytes intact, walking the source forward one codepoint at a time and writing each into the result from the end. `reverse(reverse(s)) = s` holds byte for byte.

`bal`, `any`, and `upto` still report byte positions on tagged content: a character after a multi-byte codepoint is reported at its byte offset, one or more past its codepoint index. `any` tests an isolated lead or continuation byte

against the cset rather than treating the codepoint as a unit no plain cset can represent. `detab`, `entab`, `map`, `trim`, `sort`/`sortf`, `center`, `left`, and `right` do not reorder or split multi-byte sequences, so a tagged string's encoding survives, but position, padding, and width stay byte-based. Fixing `bal`, `any`, `upto`, and `map` waits on the same codepoint-range cset as `many`.

Long strings under repeated access. Subscripting a tagged string walks from the nearer end on every access; there is no index or position cache. That suits short-lived values accessed a few times and scales poorly for a long string accessed repeatedly, such as a parser scanning a large source file. Isolated benchmarks of the walk-versus-index algorithms (not the interpreter), on a 100,000-codepoint string with mixed single- and multi-byte content, comparing a no-cache walk against a single-entry position cache plus a bucketed index:

access pattern	walk (no cache)	indexed	ratio
bursty, front-biased (typical tokenizer access)	41.9 us	0.070 us	~600x
sequential scan (typical tokenizing walk)	245.4 us	0.133 us	~1,850x

A block-based representation (tentatively `struct b_unistr`) with a single-entry position cache and an optional bucketed index, default bucket size 8, was benchmarked and not implemented. The open question is the trigger: when to promote a tagged string into that structure, and when to upgrade from the single-entry cache to the full index. A length-based threshold does not work; observed access cost during use is the more promising trigger. An optional argument to `unicode()` could also request the indexed form when the program already knows its access pattern.

Inverse of `unicode()`. Still unnamed (section 7). Same bytes, untagged descriptor. `string()` is not that operation. A content test can be derived from the inverse via `*`; a tag test cannot.

`icont` caching a codepoint count, not only the tag, for literals (section 3.1) would remove the remaining asymmetry between compile-time and runtime tagging, at the cost of the same intermediate-file width change avoided for the tag itself.

Codepoint-range character sets. Unicon's cset is a 256-bit membership table and cannot represent codepoints beyond 255. A sorted range list for codepoints above 255, leaving the existing bitmap for 0-255, uses substantially less memory across typical Unicode block compositions than the alternatives considered. It is not implemented. `many` (above) walks a tagged subject by codepoint but has no cset that can match a non-ASCII codepoint.

Pseudo-terminal `open()` still cannot carry `i`. A pty is recognized by an exact status match (`Fs_Pipe | Fs_Read | Fs_Write`); adding `Fs_Unicode` makes the match fail and the later pipe allowlist rejects the combination (error 209). Even if the match were widened, the pty branch then overwrites `status` with `Fs_Pty | Fs_Read | Fs_Write`, dropping the bit. The `read()` / `reads()` pty paths already call `UqMaybeTagRead`; the missing piece is preserving the flag through `open()`.

SFTP `open()` [3] has the same overwrite shape: after a successful SFTP open it assigns `status = Fs_SSH | ...`, which drops `Fs_Unicode`. Ordinary SSH channels use `|=` and keep the bit, so `open(..., "hi")` tags channel reads but `open(..., "hsi")` currently cannot.

Existing UTF-8 support in the standard library and tools. `uni/lib/utf8.icn` [4] walks strings byte-by-byte under the assumption that indexing is always byte-oriented. That is no longer safe for a tagged string, where indexing is codepoint-oriented. The library's tests are excluded from the suite when the feature is enabled (`tests/general/Makefile`, conditioned on `unicode` in `&features`). The library still provides codepoint-range set operations and several scanning primitives with no native equivalent, and remains usable when native tagging is off. Reconciling the two is unresolved. uscribe hit the same assumption in its LaTeX Unicode map: keys written as UTF-8 literals became tagged, so `move(*fromStr)` skipped one codepoint and left leftover bytes for `pdflatex`. That map is now built from `char()` bytes so the keys stay untagged; other byte-oriented scanners in the tree may need the same treatment.

10 Appendix: test coverage

Automated tests specific to this feature live under `tests/unicode/`, one program per concern (`size`, `subsc`, `subsc_loop`, `concat`, `convert`, `scan`, `section`, `bang`, `many`, `match`, `find`, `reverse`, `file_i`, `socket_i`, plus `ascii_*` and `languages`). `tests/general/Makefile` excludes the pre-existing `utf8`/`utf8_`

`ovld` tests (section 9) when the feature is enabled.

11 References

References

[1] Clinton Jeffery. "How to Write a Unicon Technical Report". doc/utr/utr15.tex. 2013.

[2] Unicode Consortium. "The Unicode Standard". https://www.unicode.org/versions/latest/. 2026.

[3] Jafar Al-Gharaibeh. "Native SSH and SFTP Support in Unicon". doc/utr/utr26.rst. 2026.

[4] Unicon Project. "UTF-8 class library". uni/lib/utf8.icn.